

Repurposing GUI Grounding Models for Automated UI Code Repair

Isaac Au

Carnegie Mellon University
yau@andrew.cmu.edu

Alice Le

Carnegie Mellon University
anhle@andrew.cmu.edu

Abstract

Vision-language models (VLMs) generate syntactically valid frontend code, but rendered output frequently contains visual defects the generator cannot localize from screenshots alone. We test whether GUI grounding models originally trained for web navigation can be repurposed as prompt-level scaffolding for UI code repair. We compare two grounding signals, OmniParser v2’s structural output and JEDI click-points, against an ungrounded Qwen2.5-VL- $\{7B, 72B\}$ baseline on DesignBench’s 111-sample repair task across four frameworks. Across 128 paired tests we find 27 significant gains and 10 regressions ($p < 0.05$). OmniParser on 7B Angular yields the largest effect; JEDI on 7B React halves the compile rate when the parser mismatches the dominant defect type. The pattern is consistent with a scale-dependent role: grounding scaffolds weak models, refines strong ones, and can displace attention when the model already outperforms the signal. We also document a CLIP–SSIM divergence that argues for embedding-based metrics over per-pixel structural metrics on generative UI repair.

1 Introduction

AI-powered coding agents produce syntactically valid frontend code, yet the rendered interfaces frequently contain visual defects immediately apparent to humans: misaligned spacing, poor visual hierarchy, inconsistent styling, low-contrast text, and overflowing containers. These perceptual failures arise because LLMs reason about UIs as text while never directly attending to the rendered output. Two structurally similar HTML buttons may appear indistinguishable at the token level, even when one violates basic contrast conventions (Figure 1).

Recent multimodal approaches—DOM-based inspection, screenshot-to-code pipelines, and iterative critique-and-revise loops—have attempted to close this gap but remain insufficient. Even when



Figure 1: Two structurally identical HTML buttons. The white-on-blue button is a better UI choice; the latter violates basic contrast conventions despite near-identical token-level structure.

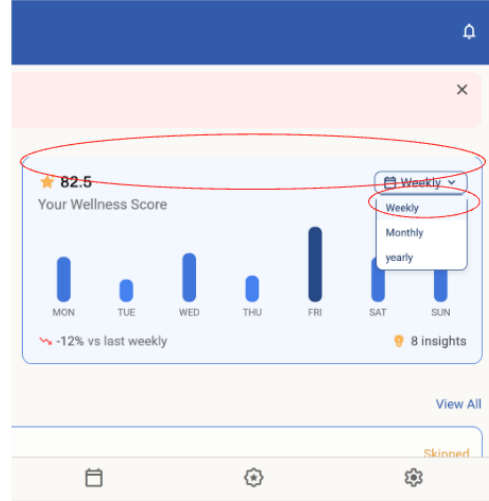


Figure 2: Claude Opus 4.6, given screenshots and input HTML, fails 3 of 3 attempts to fix missing corners and clipped edges.

given both the rendered screenshot and source code, frontier models such as Claude Opus 4.6 fail to reliably localize and repair fine-grained visual defects (Figure 2). We refer to this disconnect as the *perceptual gap* in UI code generation: the model attends to both modalities but cannot map a visual symptom (“this button is clipped”) to a code site (“adjust padding-left on selector .btn-primary”).

Crucially, structured visual understanding of interfaces already exists in a neighboring research area. GUI grounding models such as JEDI (Xie et al., 2025) and OmniParser (Yu et al., 2025),

trained for agentic web navigation, can localize interactive components, parse layout structure, and interpret visual hierarchy from screenshots. However, these capabilities have been applied almost exclusively to navigation. This motivates our central question: **can the spatial and semantic understanding that GUI grounding models acquire from navigation tasks transfer, zero-shot, to identifying and correcting visual defects in machine-generated frontend code?**

Contributions. We make three contributions.

1. **First systematic evaluation** of repurposed GUI grounding for VLM-based UI code repair. We test two grounding signals (OmniParser v2’s structural output, JEDI click-points) across two model sizes (Qwen2.5-VL- {7B, 72B}) and four web frameworks on DesignBench’s repair task. Across 128 paired tests we find 27 significant gains and 10 regressions; effects are large where the generator is weakest (7B Angular) and small but consistent where it is strong (72B visual gains across 3 of 4 frameworks).
2. **Failure-mode characterization.** We document a large regression mode—JEDI on 7B React halves the compile rate—and trace it to a mismatch between JEDI’s training distribution and the dominant defect type in that framework (alignment, on which JEDI parses only 34% of queries). Grounding can hurt as well as help.
3. **Methodology finding.** On 7B Vue, both grounding methods significantly raise high-level visual similarity and lower pixel-structural similarity on the same samples. The same divergence holds on 72B alignment defects pooled across frameworks ($N = 68$): visual similarity rises while code-edit overlap drops on the identical samples. We argue that DesignBench’s primary code-similarity metric (CMLS) systematically penalizes correct-but-different rewrites, and that CLIP is a more faithful indicator of repair quality on generative UI tasks.

2 Related Work

Our work intersects two research areas: screenshot-to-code generation with iterative visual critique and GUI grounding for navigation.

2.1 UI Code Generation and Visual Critique

Screenshot-to-code generation. Design2Code (Si et al., 2025) established the foundational task formulation and benchmark for converting webpage screenshots into HTML/CSS, curating 484 real-world webpages from the C4 dataset and introducing automatic evaluation metrics combining high-level visual similarity (CLIP) and fine-grained element matching (bounding boxes, text content, position, color). The benchmark established that single-pass generation consistently leaves visual defects.

Iterative visual critique. Two recent systems formalize the iterative-refinement insight as reinforcement learning. ReLook (Li et al., 2025) closes a generate-diagnose-refine loop in which an MLLM critic scores rendered screenshots and provides textual feedback, with GRPO optimizing the generator policy. VisRefiner (Deng et al., 2026) trains on paired visual deviations and code edits, mapping pixel-level discrepancies directly to fixes. Both confirm that visual feedback reliably improves single-pass generation. However, both ground their critique entirely in comparison against a reference design, limiting applicability when developers iterate toward novel designs.

Design-knowledge-augmented repair. DesignRepair (Yuan et al., 2024) addresses the no-reference-design case by retrieving Material Design 3 guidelines. Page extraction relies on DOM parsing and code inspection, with retrieved guidelines passed to an LLM that identifies violations and generates code edits. The system distinguishes hard constraints (usability, accessibility) from soft constraints (aesthetics). However, DOM parsing cannot detect defects that are only visible in the rendered output—layout overflow, z-index conflicts, CSS-interaction inconsistencies—which is precisely the regime where visual grounding should help.

2.2 GUI Grounding for Interface Understanding

JEDI. Xie et al. (2025) introduce a 4-million-example dataset synthesized through multi-perspective task decoupling: UI icons mined from open-source repositories, components rendered from production UI libraries (Material UI, Ant Design), and layouts sampled from operating system screens. Given a natural language instruction and

a screenshot, JEDI produces bounding-box coordinates and a description of the element’s spatial context. Multi-scale models trained on JEDI achieve state-of-the-art performance on ScreenSpot-v2 and ScreenSpot-Pro; integrating JEDI with GPT-4o as a high-level planner improved agentic task success on OSWorld from 23% to 51%.

OmniParser. OmniParser (Yu et al., 2025) takes a different approach. Given a screenshot (no query required), the v2 model uses YOLOv8 for interactable-element detection and Florence-2 for icon captioning, producing a structured element list with bounding boxes, types, and text content. We additionally couple it with EasyOCR to recover non-interactable text regions and compute pairwise geometric relations (containment, partial overlap, edge alignment, adjacency with gap distance) for each pair of detected elements. This produces a textual structural description of the interface that we can inject into the repair model’s prompt.

Other GUI grounding approaches include SeeClick (Cheng et al., 2024), which trains an end-to-end grounding model on GUI screenshots paired with natural-language references. We test JEDI and OmniParser as representatives of point-prediction and full-structural-parse styles respectively.

2.3 Trends and Gaps

Two patterns motivate this work. First, iterative visual feedback consistently improves UI code quality, but existing systems either require a reference design (ReLook, VisRefiner) or rely on DOM-based extraction that misses purely-visual defects (DesignRepair). Second, GUI grounding models possess spatial and semantic understanding of interfaces, but this understanding has been applied exclusively to navigation. The question of whether this understanding transfers to quality assessment—without retraining—remains untested.

3 Data

3.1 DesignBench

We evaluate on the *Design Repair* subtask of DesignBench (Xiao et al., 2025), a benchmark for multimodal LLMs on automated front-end engineering. The repair task provides, for each sample, (i) a rendered screenshot of a defective UI, (ii) the buggy source code, and (iii) a list of annotated visual issues. The model must output repaired code that resolves the visual defects.

DesignBench covers four front-end frameworks: React, Vue, Angular, and Vanilla HTML/CSS, and six visual defect categories: alignment, overflow, occlusion, crowding, color/contrast, and text-overlap. The repair split contains 111 samples (28 React, 27 Vue, 28 Angular, 28 Vanilla). Sample-level metadata includes a multi-defect issue list, so a single sample may contribute to multiple defect-type slices.

Why this benchmark. DesignBench is one of the few public benchmarks that pair broken-UI screenshots with both source code and annotated defect lists, with a reference-implementation evaluation pipeline that supports paired A/B testing. Each sample has a defined “broken input,” “ground-truth fix,” and rendered output, enabling controlled evaluation of grounding augmentations against an ungrounded baseline on the same inputs. Alternatives such as UICrit and WebSight evaluate generation rather than repair and lack the broken-input/fixed-output pairs we require for paired A/B testing.

Preprocessing. We use the original task as released, with one exception: we patch the React compile-detection regex in DesignBench’s evaluation pipeline to recognize nextjs-container-errors from newer Next.js versions. We otherwise reproduce the prompt template from the original paper (run_repair.py → repair_prompt.py). All experiments run with temperature 0 and seed 42 for reproducibility, in DesignBench’s both mode, which supplies both a screenshot of the page and the source code as input.

4 Method

4.1 Baseline Reproduction

Baseline choice. DesignBench evaluates nine MLLMs on the repair task. Among proprietary models, GPT-4o and Claude-3.7-Sonnet achieve the strongest performance (at the time of DesignBench’s publication). Among open-weight models, Qwen2.5-VL-72B is strongest on the repair task and is available via the same DASHSCOPE API used by the original authors, making it the most reproducible strong baseline. We additionally evaluate Qwen2.5-VL-7B because (i) it enables within-family scale analysis, and (ii) it is the base model for JEDI-7B-1080p, the grounding model in our pipeline.

Reproduction outcome. Tables 1–2 report our reproduction alongside the DesignBench paper’s reported numbers. The 72B reproduction is broadly consistent: Angular CMLS (0.676 vs. 0.631) and CMCS (0.571 vs. 0.556) are within 5 percentage points; CLIP visual similarity is comparable across all conditions. The 7B reproduction shows larger compile-rate discrepancies on Vue and Angular than the 72B. The 7B model produces borderline outputs where small perturbations can push code from “barely compiles” to “barely fails,” amplifying compile-rate differences disproportionately compared to the 72B, whose code compiles cleanly regardless. We hypothesize that closed-API check-point differences across regions and over time contribute to this gap on the 7B without affecting the 72B. We cannot verify this independently. At $N = 27$ and 28 per framework, even two or three differing samples shift the mean by several percentage points. We treat the reproduction as successful based on the close alignment of 72B results, the consistency of code-edit metrics for 7B, and visual-similarity corroboration across all conditions.

4.2 Grounded Repair Pipeline

Our pipeline (Figure 3) factors UI code repair into two stages. The grounding stage runs a frozen GUI grounding model on the broken-UI screenshot to produce a structured representation of the interface. This representation is serialized to text and injected into the repair prompt alongside the source code and the original screenshot. The repair model (Qwen2.5-VL- $\{7B, 72B\}$) then produces corrected code, which we render and compare against the target screenshot using the metrics in Table 3. The ungrounded baseline skips the grounding stage entirely, passing screenshot and source code directly to the repair model. Both grounding stage and repair stage are zero-shot, which means no additional training has been done to the models.

We compare two grounding signals.

4.2.1 OmniParser v2: Structural Grounding

OmniParser v2 (Yu et al., 2025) pairs YOLOv8 element detection with Florence-2 captioning. We extend its output with EasyOCR text regions and pairwise geometric relations to produce a single textual description of the UI structure. Specifically, the grounding output contains:

- **Elements:** every detected element, with absolute and normalized bounding box, size, area,

detection confidence, and a caption (Florence-2 for YOLO detections; the OCR string for OCR detections). Each element is assigned an integer ID.

- **Geometric relations:** for every pair of elements, we identify containment (element A fully encloses B), partial overlap, horizontal adjacency, vertical adjacency, and edge or center-line alignment. Relations are exposed as labeled facts (e.g., `contains(outer=2, inner=7)`, `adjacent_horiz(left=4, right=5, gap_px=0)`).

4.2.2 JEDI Click-Points

JEDI-7B (Xie et al., 2025) produces a single click-point per query. For each defect listed in the sample’s candidate issue list, we issue a query of the form “click on the element with $\langle$ defect type $\rangle$ ” and receive an (x, y) coordinate (or a parse failure) back. We inject the parsed coordinates into the repair prompt as `click at X, Y for  $\langle$ defect $\rangle$` . Where the parser fails, we omit the click for that defect.

This signal is intentionally narrower than OmniParser’s. JEDI was trained for interactable-element clicking, so its parse rate varies by defect type: 74% on crowding, 58% on overflow, 40% on occlusion, 36% on color/contrast, 34% on alignment, 33% on text-overlap. Frameworks with defect distributions skewed toward poor-parse types (e.g., React with alignment-heavy defects) carry corrupted prompts.

4.3 Implementation

The repair model is Qwen2.5-VL accessed via the international Dashscope endpoint, with temperature 0 and seed 42, identical to the original DesignBench setup. The OmniParser caches and JEDI caches are computed once and shared between 7B and 72B repair runs (the grounding signal is independent of the repair-stage model). OmniParser runs on a single A100 ($\sim 0.6s$ /screenshot at full resolution); JEDI runs via vLLM on a single A100 ($\sim 0.5s$ /click). The grounding caches are $<1MB$ total across the 111 samples.

4.4 Evaluation Metrics

We report the five DesignBench metrics plus three rendered-screenshot metrics, mapped to repair-pipeline stages (Table 3).

Statistical testing. For each (model, signal, framework, metric) cell, we compare grounded out-

FW	N	CSR	CMLS (paper / ours)	CMCS	IssAcc	CLIP (ours)
React	28	0.929 / 0.857	0.634 / 0.339	0.542 / 0.230	0.395	0.771
Vue	27	1.000 / 1.000	0.509 / 0.213	0.392 / 0.143	0.213	0.796
Angular	28	0.929 / 0.964	0.676 / 0.631	0.571 / 0.556	0.379	0.821
Vanilla	28	— / 1.000 [†]	0.609 / 0.532	0.586 / 0.510	0.369	0.791
Overall	111	0.952 / 0.955	0.607 / 0.430	0.523 / 0.360	0.340	0.795

Table 1: Qwen2.5-VL-72B repair baseline. CSR, CMLS, CMCS shown as paper / ours; IssAcc and per-framework CLIP are ours only. [†]Vanilla CSR is computed under a patched detection rule: DesignBench’s evaluator returns a non-NULL `compile_error` for successfully rendered Vanilla samples, which we treat as compile-successful.

FW	N	CSR	CMLS (paper / ours)	CMCS	IssAcc	CLIP (ours)
React	28	0.286 / 0.250	0.362 / 0.182	0.257 / 0.139	0.345	0.632
Vue	27	0.111 / 0.926	0.245 / 0.237	0.180 / 0.179	0.210	0.785
Angular	28	0.036 / 0.571	0.641 / 0.304	0.577 / 0.206	0.173	0.486
Vanilla	28	— / 1.000 [†]	0.499 / 0.422	0.478 / 0.394	0.345	0.796
Overall	111	— / 0.687	0.437 / 0.287	0.373 / 0.230	0.269	0.674

Table 2: Qwen2.5-VL-7B repair baseline. Format matches Table 1. [†]Vanilla CSR uses the patched detection rule described in Table 1.

Metric	Measures	Stage
CSR	Compile success	Generation
IssAcc	Correct defect-type labels	Identification
CMLS	Jaccard sim. of AST edits	Localization
CMCS	CMLS $\times$ CodeBLEU of edits	Loc. + Content
CodeS	String similarity	Surface
CLIP	Render embedding sim.	Visual semantic
SSIM	Render pixel-structural sim.	Visual pixel
MAE	Render mean abs. pixel error	Visual pixel

Table 3: Repair metrics mapped to pipeline stages. Rendered-output metrics (CLIP, SSIM, MAE) are computed on the rendered output of each sample’s generated code, against the ground-truth render.

put to ungrounded baseline using a paired signed-rank test (two-sided) on continuous metrics and a exact binomial test on discordant pairs for the binary compile-rate metric. We use $\alpha = 0.05$ throughout. We do not apply multiple-comparison correction; we report all 128 tests in the panel and discuss this choice in §8. Per-defect-type analysis pools samples across frameworks ($N = 11\text{--}68$ per defect type, with multi-defect samples counted once per defect).

5 Results

5.1 Summary of results

Across our $2 \times 2 \times 4$ panel (2 model sizes, 2 grounding signals, 4 frameworks) we find 27 statistically significant gains and 10 significant regressions ($p < 0.05$, paired Wilcoxon). The pattern is structured by model size: 7B shows large effects on the weakest cell and large regressions on a

mismatched cell; 72B shows small but consistent gains across most frameworks with one significant regression.

Largest effect: 7B + OmniParser on Angular. Visual similarity (CLIP, SSIM), code-edit content (CMCS), and compile rate all rise significantly on this cell (Table 4). This is the cell where the baseline Qwen-7B performs worst—Angular requires coordinated edits across separate `template.html` and `component.ts` files, and 43% of samples fail to compile without help. OmniParser’s element list, OCR text, and geometric relations supply the spatial scaffolding the weak model lacks. Five additional samples compile under grounding.

Largest regression: 7B + JEDI on React. Compile rate halves and every visual and AST metric drops significantly on this cell (Table 4). React samples are dominated by alignment defects, and JEDI’s parse rate on alignment is only 34%. The resulting prompt contains many unparseable JEDI outputs (empty or malformed coordinate strings), which the 7B model follows into malformed JSX. The signal mismatch between JEDI’s training distribution (interactable-element clicking) and the dominant defect type in this cell turns a helpful augmentation into a harmful one.

5.2 Cross-Framework, Cross-Method Panel

Table 4 reports significant cells for the 7B model; Table 5 for the 72B. Cells are highlighted green

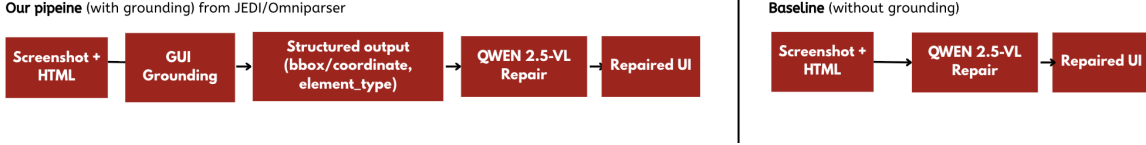


Figure 3: Our grounded repair pipeline (left) versus the ungrounded baseline (right). Our pipeline inserts a frozen GUI grounding model (JEDI-7B-1080p or OmniParser v2) between the broken-UI screenshot and the repair model. The grounding model emits a structured output (bounding boxes, click-point coordinates, element types, captions), which is serialized to text and injected into the repair prompt alongside the original HTML. Qwen2.5-VL produces repaired code that we render into a new screenshot. The baseline omits the grounding stage. All other prompt structure, decoding parameters, and evaluation code are identical across the two conditions.

for significant gains, red for significant regressions, and dashed (—) where the comparison was not significant. We restrict to both-mode results in the main paper.

Three patterns are visible.

Scale changes role. On 7B, OmniParser’s effect is concentrated on the worst-performing framework (Angular). On 72B, OmniParser produces small but consistent CLIP gains across three frameworks (Vue, Angular, Vanilla). The 72B effect is roughly $8\times$ smaller than the 7B Angular effect on CLIP, but appears across more cells. Grounding has different roles based on the model. For the weak model, it acts as scaffolding to identify structural issues and improve code generation; on the strong model, grounding helps identify weak spots the model may have missed on the first pass.

JEDI hits diminishing returns on the strongest cell. On 72B Angular, where the baseline is already the strongest cell in our entire panel (code-edit overlap 0.63, visual similarity 0.82, compile rate 0.96), JEDI sharpens pixel-level fidelity but produces a small offsetting slip on embedding and structural similarity. The visual-metric movements are an order of magnitude smaller than the 7B Angular gains. We interpret this cell as the boundary where grounding hits diminishing returns: when the base model already handles the task well, additional spatial hints contribute little marginal information and the small displacement of the model’s own attention shows up as a modest visual-similarity slip.

CLIP and SSIM disagree. On 7B Vue, both grounding methods produce significant CLIP gains and significant SSIM losses on the same samples (Table 4). CLIP embedding similarity rewards high-level visual match; SSIM and MAE reward pixel preservation. Grounding causes the model to re-layout the page in a way that fixes the defect (CLIP

up) but moves pixels around (SSIM down).

5.3 Per-Defect Slice

Pooling samples across all four frameworks by defect type ($N = 11\text{--}68$), 72B + OmniParser on alignment defects ($N = 68$, the largest category) shows simultaneous CLIP $+0.007$ ($p < 0.01$), CMLS -0.055 ($p < 0.05$), and CMCS -0.045 ($p < 0.05$) on the *same* samples. Because alignment defects have no single AST representation, grounding helps the model fix the layout visually while the resulting AST diverges from the reference implementation.

5.4 Qualitative Examples

Qualitative Examples can be found in Figure 4 in Appendix, illustrating practical effects of JEDI and OmniParser v2 grounding.

6 Discussion

6.1 What the pattern means

The 7B-vs-72B asymmetry is consistent with the pattern that grounding appears to help when the model’s internal spatial understanding is worse than the grounding signal’s, and to hurt when the model’s understanding is already better. On 7B Angular, the baseline fails on 43% of samples because the generator cannot map screenshot regions to multi-file code locations; OmniParser’s structured element list closes that gap. On 72B Angular, the baseline already handles spatial coordination well, and JEDI’s click coordinates appear to displace useful attention.

This is the same pattern documented in distillation and chain-of-thought literature: explicit structure helps weak models more, but strong models can be hurt by redundant or noisy explicit signals.

Metric	Angular		React		Vue		Vanilla	
	+Omni	+JEDI	+Omni	+JEDI	+Omni	+JEDI	+Omni	+JEDI
CLIP	+.141**	—	—	−.309**	+.021**	+.012*	—	—
SSIM	+.111**	—	—	−.304**	−.016**	−.043*	—	—
MAE	—	—	—	artifact†	—	+4.78*	—	—
CMLS	—	—	—	−.096**	—	—	—	—
CMCS	+.073*	—	—	−.084**	—	—	—	—
CSR	—	—	—	−.500**	—	—	—	—
IssAcc	—	+.256**‡	—	—	—	+.469**‡	—	+.310**‡

Table 4: Qwen2.5-VL-7B grounded vs. ungrounded (Δ on each metric). * $p < 0.05$, ** $p < 0.01$. †React 7B+JEDI MAE is artifactual: 14 samples have MAE= 0 because the page failed to compile and rendered as blank, biasing the mean downward. ‡JEDI IssAcc gains discussed in §8.

Metric	Angular		React		Vue		Vanilla	
	+Omni	+JEDI	+Omni	+JEDI	+Omni	+JEDI	+Omni	+JEDI
CLIP	+.009*	−.013**	—	—	+.012**	+.015*	+.018**	+.013**
SSIM	+.003*	−.026*	—	+.011**	—	+.012**	+.019**	+.026*
MAE	—	−3.04**	—	−1.15**	—	−.25**	−.34*	—
IssAcc	—	+.225**‡	—	+.270**‡	—	+.441**‡	—	+.238**‡

Table 5: Qwen2.5-VL-72B grounded vs. ungrounded. Significance markers and IssAcc caveat as in Table 4.

6.2 What grounding actually does

The visual-vs-code-edit divergence on 72B alignment ($N = 68$) tells us what grounding changes about the repair behavior. With grounding, the same 72B model produces visually-better outputs that are more different from the reference implementation. Reference-based metrics penalize this behavior while rendered-output metrics reward it. The implication is that DesignBench’s primary code metric, CMLS, is not a faithful measure of repair quality on a generative task: a model that fixes the defect with different code is penalized for the difference. More broadly, our results are consistent with the view that the value of explicit structured context to a generative model scales inversely with the model’s intrinsic competence on the structure being supplied.

6.3 Strengths vs. baselines

The OmniParser-grounded 7B variant outperforms the ungrounded 7B baseline on every visual metric on Angular, and on high-level visual similarity for 72B Vue, Angular, and Vanilla. JEDI provides additional small-but-consistent pixel-level improvements on 72B for non-Angular frameworks. Both grounding methods raise defect-identification accuracy (IssAcc); we discuss the interpretation of the JEDI gains in §8.

6.4 Surprising findings

The methodology finding. We did not expect both grounding methods to show simultaneous CLIP gain and SSIM loss on 7B Vue. The fact that the same divergence appears across two unrelated grounding methods on the same cell, plus on the 72B alignment slice, makes us treat it as a property of the task rather than the methods. This affects how future work on grounded UI repair should be evaluated.

6.5 Practical implications

Choose grounding by signal-to-task fit. The grounding signal must match the defect distribution. JEDI’s clicking-trained outputs work well when the defect is anchored on a discrete element (buttons, icons) and fail when the defect is a property of the layout (alignment, overflow). OmniParser’s structured output covers more defect types but is heavier in the prompt.

Use grounding where the model is weakest. The 7B Angular gain—+14 CLIP points—is much larger than any 72B effect we observed. Practitioners should prioritize grounding for cells where the baseline model is weak on the target framework, rather than applying it uniformly.

Report rendered-output similarity (CLIP), not code-edit overlap (CMLS). For generative UI repair tasks, code-similarity metrics systematically penalize correct-but-different rewrites. CLIP is

closer to “does the output look like the target” and tracks repair quality more faithfully. We recommend CLIP as the primary metric and CMLS as a diagnostic for “did the model edit the same code.”

7 Conclusion and Future Work

We tested whether GUI grounding models, originally trained for web navigation, can transfer zero-shot to UI code repair when their output is injected as structured prompt context to a vision-language repair model. On DesignBench’s 111-sample repair task across four frameworks and two model sizes, we find 27 statistically significant gains and 10 significant regressions. The largest gain is OmniParser on 7B Angular; the largest regression is JEDI on 7B React, where compile rate halves. The pattern is consistent with a scale-dependent role: grounding appears to scaffold weak models on hard cells, refine strong models on easier cells, and is associated with regression when the model is already stronger than the grounding signal.

The contribution beyond a single benchmark cell is the demonstration that the role of grounding depends on the relative quality of the generator and the grounding signal. This frames future work on grounded VLM-based code generation: the question is not “does grounding help?” but “where does grounding help, and why?”

Future work.

- **Hybrid OmniParser + JEDI.** On 7B Angular, OmniParser contributes visual-similarity gains; JEDI contributes defect-identification gains. The strengths are complementary. A principled combination that uses OmniParser for spatial scaffolding and JEDI for attention-anchoring on parsed defects is the most promising single follow-up.
- **LoRA fine-tuning of the grounding model.** If the zero-shot transfer pattern holds, lightweight adaptation on UI defect data could close the remaining gap and produce a defect-aware grounding model usable across the field. We did not pursue this in the current scope.
- **Smaller and larger models.** The two-point trend (7B vs. 72B) is suggestive but not conclusive. A 3B model and a larger frontier checkpoint would establish whether the “scaffolding-vs-displacement” story is monotonic in scale.

8 Limitations

Sample size. DesignBench’s 111-sample repair task gives $N = 27$ or 28 per framework and $N = 11$ to 68 per defect-type slice. Paired Wilcoxon is appropriate at this N , though the sample size limits the precision of effect-size estimates.

Single benchmark. We evaluate on DesignBench only. UICrit, WebSight, and real-world broken-UI corpora may exhibit different defect distributions or task difficulties. The “grounding scaffolds weak models” pattern should be tested on additional benchmarks before being treated as a general finding.

Single model family. We test only Qwen2.5-VL- $\{7B, 72B\}$. Whether the scale-dependent pattern holds for GPT-4o, Claude, or LLaVA-family models is open.

JEDI defect-identification interpretation. JEDI’s prompt format includes the defect type as part of the click query (e.g., “click on the element with alignment”), so the JEDI defect-identification gains may partially reflect the prompt format rather than improved defect identification. We do not center this metric in JEDI claims. OmniParser’s prompt does not name defect types and is unaffected.

Temperature 0 may hide variance. We use temperature 0 for reproducibility, which means each cell represents a single greedy decode rather than a distribution. Some cells likely sit near the “helpful vs. harmful” decision boundary and could swing under non-zero temperature.

9 Ethical Considerations

Visual quality and accessibility. A subset of UI defects we evaluate—color/contrast violations in particular—are accessibility issues for users with visual impairments. A repair pipeline that “fixes” contrast by removing the content is worse than the original defect because it removes information. We flag this as a real risk: any deployed repair system must validate that fixes do not eliminate accessibility-relevant content. We recommend that production deployments combine this kind of automated repair with explicit accessibility validation rather than treating automated visual-similarity scores as sufficient.

Distributional bias in DesignBench. DesignBench’s 111 repair samples are curated from a fixed set of broken-UI patterns. The defect distribution skews toward alignment and crowding, with text overlap underrepresented ($N = 3$). Conclusions about which grounding methods work best for which defect types should be qualified by this distribution. A model evaluated only on DesignBench may underperform on real-world defect distributions.

Reproducibility risk via API drift. We used Alibaba’s Dashscope endpoint. Closed-API model endpoints can change checkpoint versions silently across regions and over time, which affects reproducibility. We have flagged in the baseline reproduction that 7B compile rate diverges substantially from the original paper, and we hypothesize that regional/temporal checkpoint differences underlie the divergence. Our findings rely on within-experiment paired comparisons (grounded vs. ungrounded on the same checkpoint at the same time), so the within-paper claims are robust to API drift, but anyone reproducing our work months later may see different absolute numbers.

Building responsibly. Researchers building on this work should (i) report rendered-output metrics alongside code-similarity metrics; (ii) test grounding augmentations on multiple model sizes before claiming generality; (iii) verify that grounded repairs preserve accessibility-relevant content rather than removing it; and (iv) treat grounding-injection prompts as failure-prone—the React + JEDI catastrophe in our paper is a concrete example of how a plausible engineering choice can halve compile rates.

References

- Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu. 2024. [SeeClick: Harnessing GUI grounding for advanced visual GUI agents](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics.
- Jie Deng, Kaichun Yao, and Libo Zhang. 2026. [VisRefiner: Learning from visual differences for screenshot-to-code generation](#). *Preprint*, arXiv:2602.05998. Preprint.
- Yuhang Li, Chenchen Zhang, Ruilin Lv, Ao Liu, Ken Deng, Yuanxing Zhang, Jiaheng Liu, Wiggin Zhou, and Bo Zhou. 2025. [ReLook: Vision-grounded RL with a multimodal LLM critic for agentic web coding](#). *Preprint*, arXiv:2510.11498. Preprint.
- Chenglei Si, Yanzhe Zhang, Ryan Li, Zhengyuan Yang, Ruibo Liu, and Diyi Yang. 2025. [Design2Code: Benchmarking multimodal code generation for automated front-end engineering](#). *Preprint*, arXiv:2403.03163. Preprint.
- Jingyu Xiao, Ming Wang, Man Ho Lam, Yuxuan Wan, Junliang Liu, Yintong Huo, and Michael R. Lyu. 2025. [DesignBench: A comprehensive benchmark for MLLM-based front-end code generation](#). *Preprint*, arXiv:2506.06251. Preprint.
- Tianbao Xie, Jiaqi Deng, Xiaochuan Li, Junlin Yang, Haoyuan Wu, Jixuan Chen, Wenjing Hu, Xinyuan Wang, Yuhui Xu, Zekun Wang, Yiheng Xu, Junli Wang, Doyen Sahoo, Tao Yu, and Caiming Xiong. 2025. [Scaling computer-use grounding via user interface decomposition and synthesis](#). *Preprint*, arXiv:2505.13227. Preprint.
- Wenwen Yu, Zhibo Yang, Jianqiang Wan, Sibao Song, Jun Tang, Wenqing Cheng, Yulian Liu, and Xiang Bai. 2025. [OmniParser v2: Structured-points-of-thought for unified visual text parsing and its generality to multimodal large language models](#). *Preprint*, arXiv:2502.16161. Preprint.
- Mingyue Yuan, Jieshan Chen, Zhenchang Xing, Aaron Quigley, Yuyu Luo, Tianqi Luo, Gelareh Mohammadi, Qinghua Lu, and Liming Zhu. 2024. [DesignRepair: Dual-stream design guideline-aware front-end repair with large language models](#). *Preprint*, arXiv:2411.01606. Preprint.

A JEDI Parse Rate by Defect Type

JEDI’s parse rate is highly defect-type dependent: crowding 74%, overflow 58%, occlusion 40%, color/contrast 36%, alignment 34%, text-overlap 33%. The 7B React large regression is consistent with React’s defect distribution being alignment-heavy and JEDI’s alignment parse rate being among its lowest.

B Qualitative Examples

Figure 4 shows two paired before/after comparisons that concretize the grounded-vs-ungrounded difference for each grounding signal in a cell where it is effective.

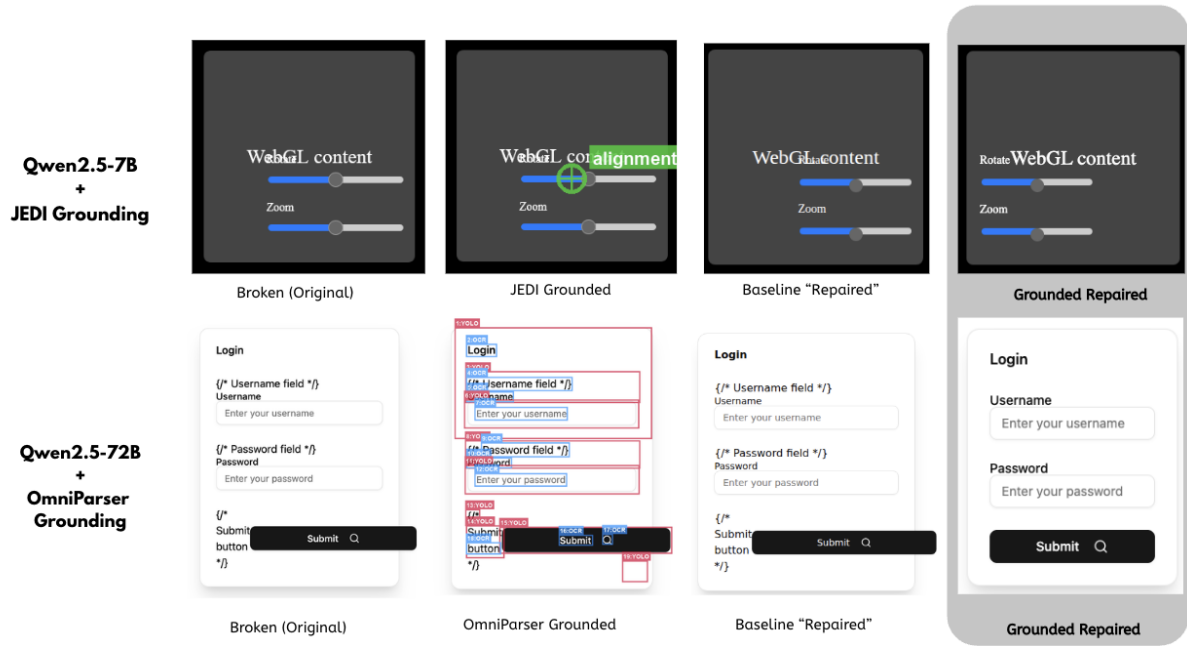


Figure 4: Paired qualitative comparison of grounded vs. ungrounded repair on two representative cells. Each row shows, left to right: the broken input, the grounding model’s output overlaid on the screenshot, the ungrounded baseline’s repair, and the grounded repair. **Top:** Qwen2.5-VL-7B with JEDI grounding on a WebGL slider UI. JEDI marks the misaligned region with an “alignment” click target (column 2). The ungrounded baseline (column 3) reproduces the broken layout almost verbatim, while the grounded repair (column 4) aligns the slider label. **Bottom:** Qwen2.5-VL-72B with OmniParser structural grounding on a Login form. The broken input contains template placeholder strings (`{/* Username field */}`, `{/* Submit button */}`) leaking through to the rendered output. OmniParser detects every leaked placeholder via YOLO + OCR (column 2), each as a distinct text region. The ungrounded baseline (column 3) preserves the placeholder strings unchanged; the grounded repair (column 4) removes them and produces a clean form. The two cases illustrate the complementary roles of the two grounding signals: JEDI provides a focused click target on a single defective region, while OmniParser exposes the full element list including non-interactable text artifacts that neither baseline nor JEDI surfaces.